

Fraud Detection in Electricity & Gas Consumption - Notebook Guide

Project Overview

This notebook implements a machine learning system to detect fraudulent electricity and gas consumption patterns. Using customer billing data and invoice history, we develop predictive models to identify high-risk customers for investigation, achieving 65% fraud detection by reviewing only 25% of customers with a 124% ROI.

How to Run in Google Colab

1. **Upload the notebook** to Google Colab
 2. **Upload datasets:** `client.csv` and `invoice.csv` to Colab's `/content/` directory or mount Google Drive
 3. **Run all cells sequentially** (Runtime → Run all)
 4. **Expected runtime:** 3 hours with CPU, 60-80 minutes with GPU
-

Notebook Structure

Steps 1-2: Setup & Data Loading

Import required libraries (pandas, numpy, sklearn, matplotlib) and load client and invoice datasets.

Steps 3-5: Data Inspection

Examine dataset structure, dimensions, data types, and understand the relationship between customer and invoice tables.

Steps 6-13: Exploratory Data Analysis (EDA)

- Analyze fraud distribution (4.39% fraud rate, 22:1 imbalance)
- Explore geographic patterns and customer categories
- Compare fraud vs legitimate consumption patterns
- Identify correlations and outliers

Key Finding: Fraudsters consume 21% less on average; Category 11 has 6.9x higher fraud rate.

Steps 14-15: Data Cleaning & Feature Engineering

- Handle outliers, missing values, and temporal features
- Aggregate invoice-level data to customer-level
- Create 110 features: consumption statistics, ratios, behavioral flags, categorical encodings

Step 16: Data Preparation

- Encode categorical variables
- Handle missing/infinite values
- Scale features using StandardScaler
- Perform stratified train-test split (80/20)

Step 17: Baseline Models

Establish performance benchmarks with Majority Class predictor and Logistic Regression (AUC=0.775, F1=0.163).

Step 18: Advanced Models with Class Balancing

Train Random Forest, XGBoost, and RF+SMOTE with class balancing techniques. Apply threshold tuning to optimize precision-recall tradeoff.

Best Result: RF Balanced (AUC=0.788, F1=0.211, Recall=55.3%)

Step 19: Hyperparameter Tuning

- RandomizedSearchCV (50 combinations, 5-fold CV)
- GridSearchCV for fine-tuning
- XGBoost hyperparameter optimization

Outcome: Validated manual tuning; XGBoost improved +0.032 AUC.

Step 20: Advanced Techniques - LightGBM & Isolation Forest

Test research-inspired approaches including LightGBM and hybrid anomaly detection.

Result: No improvement over Random Forest (AUC dropped to 0.770).

Step 21: Feature Selection

Test aggressive feature reduction (110 → 30/40/50 features).

Result: All features contribute; removal hurt performance.

Step 22: Temporal Feature Engineering

Create 11 time-based features from invoice patterns (trends, volatility, billing consistency).

Result: Insufficient invoice history; AUC dropped to 0.776.

Step 23: Ensemble Modeling

Develop Voting Classifier combining Random Forest, XGBoost, and Logistic Regression with [2,1,1] weights.

Winner: Voting Ensemble (AUC=0.792, F1=0.191, Recall=65.0%) ★

Step 24: Final Model Comparison

Compare all 8 models tested. Voting Ensemble achieves highest ROC-AUC (0.792).

Step 25: Risk Tier System

Implement 5-tier risk classification (Critical/High/Medium/Low/Very Low) with actionable investigation protocols.

Business Impact:

- Investigate 25.4% of customers (Critical + High tiers)
- Catch 65% of frauds
- ROI: 124% (\$185K net benefit)

Step 26: Publication-Quality Visualizations

Generate 8 figures for reporting:

1. ROC curves comparison
2. Precision-Recall curves
3. Performance metrics (4-panel)
4. Confusion matrices (top 3 models)
5. Feature importance (top 20)
6. Risk tier system overview (4-panel)
7. Business impact analysis
8. Radar chart comparison

All figures saved at 300 DPI in working directory.

Key Outputs

- **Final Model:** Voting Ensemble (saved as .pk1)
 - **Performance:** AUC=0.792, F1=0.191, Recall=65.0%
 - **Business Value:** 124% ROI, \$185K net savings
 - **Visualizations:** 8 publication-ready figures
-

Requirements

pandas, numpy, scikit-learn, xgboost, lightgbm,
matplotlib, seaborn, imbalanced-learn

Install missing packages: `!pip install lightgbm imbalanced-learn`

Total Steps: 26 | **Runtime:** ~3 hours | **Models Tested:** 8 | **Final AUC:** 0.792